Лабораторна 1.7

Блоки та стратегії розподілу пам’яті

Виконав: Кузьменко Віталій Юрійович, група Бд-F2-26-3-ПІ. Файл: memory.cpp. Варіант у цій роботі не використовується.

C++17 погоджено через обов’язкові ::, new та delete. У C++17 ключове слово auto виводить тип, тому автоматична змінна з прикладу оголошена як int iB = 0. Вона має той самий потрібний час існування.

Перевірка: G++ 16.2.0, GDB 18.1, Windows x64; -O0 -g3 -Wall -Wextra -Wpedantic -Werror. Компіляція без попереджень. Звичайний запуск та окремий режим --leak-demo завершилися з кодом 0.

Статична пам’ять та покрокове виконання

У GDB виконано breakpoint main, watch nStaticGlobal, display, next та зупинки у позначених WATCH_* місцях. Hardware watchpoint зафіксував зміну nStaticGlobal: 10 -> 15. Після кроку ::nGlobal = ::nGlobal * 2 глобальна змінна стала 90; локальна nGlobal залишилася 10.

Область	Спостережені значення	Пояснення
main	nValue=1; nOther=2, далі 3	Зовнішня змінна доступна у вкладених блоках.
Блок 1.1	nValue=11; static=5, далі 6	Локальне ім’я перекриває nValue з main.
Блок 1.1.1	nValue=103; nOther=3; static=6	Ще один локальний nValue; nOther змінено назовні.
Блок 1.2	nValue=23	Незалежна змінна послідовного блоку.
Повернення в main	nValue=1; nOther=3	Перекриття припинилося; зовнішнє значення збереглося.

У циклі static iA створюється один раз, а автоматична iB ініціалізується нулем при кожному вході. Спостережено:

Ітерація i	iA після +1	iB після +1
0	1	1
1	2	1
2	3	1
3	4	1
4	5	1

Адреса iA в одному запуску була незмінною: 0x7ff76809f080. Адреса iB теж повторювалася (0x3e41ff998), але щоразу створювався новий автоматичний об’єкт. Рівність адрес не доводить однаковий час існування.

Купа, час існування і втрата адреси

Для pnFirst, pnSecond, pnLocal і pnSaved у GDB досліджено &покажчик, покажчик і *покажчик. Це три різні величини: адреса змінної-покажчика, адреса об’єкта у купі та його значення.

Зупинка	Спостереження
WATCH_HEAP1	&pnFirst=0x3e41ff9a8; pnFirst=0x1658556e1e0; *pnFirst=25
WATCH_HEAP2	pnFirst=0; pnSecond=0x1658556e1e0; *pnSecond=50
Вкладений блок	pnLocal та pnSaved містять 0x1658556e1e0; значення 75
Після вкладеного блоку	pnSaved=0x1658556e1e0; *pnSaved=75
Після delete і занулення	pnFirst=0; pnSecond=0; pnSaved=0

Після виходу з блоку зникає автоматичний покажчик pnLocal, але виділений new об’єкт живе до delete. Якщо адресу збережено в pnSaved, її можна використати та звільнити пам’ять поза блоком. Алокатор у цьому запуску повторно використав ту саму адресу для кількох послідовних об’єктів.

Навмисні витоки в окремому запуску

Режим memory.exe --leak-demo досліджено окремим процесом GDB. Виконано next на операторі втрати та перезапису адреси; не розіменовували звільнену пам’ять.

Сценарій	Фактичне спостереження	Причина витоку
Занулення	pnLost: 0x2655037bed0 -> 0	Втрачено єдину адресу int зі значенням 111.
Вихід із блоку	pnLocal=0x26550383440; *pnLocal=222	Покажчик локальний; delete відсутній.
Перезапис адреси	0x26550383480 (*=333) -> 0x2655037bbc0 (*=444)	Новий new замінив адресу першого об’єкта; звільнено лише другий.

У цьому режимі навмисно лишаються три об’єкти int: 12 байтів корисних даних, без урахування накладних витрат алокатора. Ця кількість випливає з виконаних new і delete; окремим детектором витоків її не вимірювали. Після завершення процесу ОС повертає його пам’ять.

Після delete адреса сама не занулюється: nullptr присвоєно явно. Подвійний delete, читання після delete і невідповідна пара new[]/delete є прикладами невизначеної поведінки; у коді вони пояснені, але не виконуються.

Межі перевірки

Дослідження виконане у GDB через display/print, breakpoint, hardware watchpoint і next. Графічне вікно Watch IDE не перевірено. Для його демонстрації поставити зупинки WATCH_LOOP, WATCH_HEAP1 та WATCH_DELAYED_OUT і додати зазначені вирази в Watch. Жодних скриншотів або непроведених вимірювань у результатах немає.